Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Проверил:
Доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии Воронкин Р.А

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Введение в нейронные сети. Построение и обучение базовых моделей на Python с использованием TensorFlow и PyTorch.

Цель работы: Изучение основ построения нейронных сетей и получение практических навыков работы с фреймворками TensorFlow и PyTorch.

Репозиторий: https://github.com/bickrosss/NN_lab1

Порядок выполнения работы:

Задание: Создайте систему компьютерного зрения, которая будет определять тип геометрической фигуры. Используя подготовленную базу и шаблон ноутбука проведите серию экспериментов по перебору гиперпараметров нейронной сети, распознающей три категории изображений (треугольник, круг, квадрат).

1. Поменяйте количество нейронов в сети, используя следующие значения:

- один слой 10 нейронов
- один слой 100 нейронов
- один слой 5000 нейронов.

2. Поменяйте активационную функцию в скрытых слоях с relu на linear.

3. Поменяйте размеры batch_size:

- 10
- 100
- 1000

4. Выведите на экран получившиеся точности.

Всего должно получиться 18 комбинаций указанных параметров.

Создайте сравнительную таблицу по результатам проведенных тестов.

Решение:

1. Реализовали систему компьютерного зрения для классификации геометрических фигур: треугольников, кругов и квадратов. Сперва импортируются необходимые библиотеки, включая TensorFlow для

построения нейронных сетей и NumPy для обработки массивов. Далее выполняется загрузка архива с датасетом из облачного хранилища и его распаковка непосредственно в рабочую среду для дальнейшего использования.

На этапе подготовки данных формируется обучающая выборка. Программа обходит папки с изображениями, считывает файлы и приводит их к единому размеру 20 на 20 пикселей в оттенках серого. На основе названий папок формируются метки классов. Затем выполняется нормализация данных: значения пикселей переводятся в диапазон от 0 до 1, а двумерные изображения разворачиваются в плоские векторы длиной 400 элементов. Метки классов преобразуются в формат one-hot encoding для корректной работы функции потерь.

Основную часть исследования посвятили автоматизированному перебору гиперпараметров нейронной сети. В цикле последовательно создаются и обучаются 18 различных конфигураций моделей, где варьируются количество нейронов в скрытом слое (10, 100 или 5000), функции активации (relu или linear) и размер пакета данных при обучении (10, 100 или 1000). Каждая модель обучается в течение 10 эпох с использованием оптимизатора Adam и функции потерь categorical crossentropy, после чего фиксируется итоговая точность на обучающей выборке.

Завершается работа сведением всех полученных результатов в единую аналитическую таблицу с помощью библиотеки pandas. Это позволяет наглядно сравнить эффективность различных комбинаций параметров. По итоговым данным видно, как изменение структуры сети и режима подачи данных влияет на точность распознавания фигур, что помогает определить оптимальную архитектуру для решения данной задачи классификации.

```python
# Нормализация пикселей
x_train = x_train.astype('float32') / 255.0

# Разворачиваем изображения в вектор: из (302, 20, 20, 1) в (302, 400)
x_train_flat = x_train.reshape(x_train.shape[0], -1)

# Преобразование меток в one-hot encoding (3 класса)
num_classes = 3
y_train_cat = utils.to_categorical(y_train, num_classes)

print('x_train_flat shape:', x_train_flat.shape)
print('y_train_cat shape:', y_train_cat.shape)
```

```
x_train_flat shape: (302, 400)
y_train_cat shape: (302, 3)
```

Рисунок 1. Норализация

```python
# Параметры для перебора
neurons_list = [10, 100, 5000]
activations  = ['relu', 'linear']
batch_sizes  = [10, 100, 1000]

results   = []
input_dim = x_train_flat.shape[1]  # 400
epochs    = 10

combo_num = 0
for neurons in neurons_list:
    for activation in activations:
        for batch_size in batch_sizes:
            combo_num += 1

            # Создание модели
            model = Sequential([
                Dense(neurons, activation=activation, input_shape=(input_dim,)),
                Dense(num_classes, activation='softmax')
            ])

            model.compile(
                optimizer=Adam(),
                loss='categorical_crossentropy',
                metrics=['accuracy']
            )

            history = model.fit(
                x_train_flat, y_train_cat,
                epochs=epochs,
                batch_size=batch_size,
                verbose=0
            )

            final_acc = history.history['accuracy'][-1]
            print(f'[{combo_num}/18] Нейронов={neurons}, Активация={activation}, Batch={batch_size}  →  Точность: {final_acc:.4f}')

            results.append({
                'Нейронов': neurons,
                'Активация': activation,
                'Batch size': batch_size,
                'Точность': round(final_acc, 4)
            })
```

```
[1/18] Нейронов=10, Активация=relu, Batch=10  →  Точность: 0.8344
[2/18] Нейронов=10, Активация=relu, Batch=100  →  Точность: 0.6424
[3/18] Нейронов=10, Активация=relu, Batch=1000  →  Точность: 0.4371
[4/18] Нейронов=10, Активация=linear, Batch=10  →  Точность: 0.7152
[5/18] Нейронов=10, Активация=linear, Batch=100  →  Точность: 0.5695
[6/18] Нейронов=10, Активация=linear, Batch=1000  →  Точность: 0.4570
[7/18] Нейронов=100, Активация=relu, Batch=10  →  Точность: 0.8874
[8/18] Нейронов=100, Активация=relu, Batch=100  →  Точность: 0.7914
[9/18] Нейронов=100, Активация=relu, Batch=1000  →  Точность: 0.7483
[10/18] Нейронов=100, Активация=linear, Batch=10  →  Точность: 0.7980
```

Рисунок 2. Параметры для перебора

```python
# Сравнительная таблица результатов
df = pd.DataFrame(results)
df.index = range(1, len(df) + 1)
df.index.name = ''

print('Сравнительная таблица результатов:')
print(df.to_string())
```

```
Сравнительная таблица результатов:
    Нейронов Активация  Batch size  Точность
1         10      relu          10    0.8245
2         10      relu         100    0.5464
3         10      relu        1000    0.5464
4         10    linear          10    0.8278
5         10    linear         100    0.5762
6         10    linear        1000    0.5497
7        100      relu          10    0.9338
8        100      relu         100    0.7450
9        100      relu        1000    0.7384
10       100    linear          10    0.8808
11       100    linear         100    0.7450
12       100    linear        1000    0.5993
13      5000      relu          10    0.9007
14      5000      relu         100    0.8079
15      5000      relu        1000    0.4570
16      5000    linear          10    0.6623
17      5000    linear         100    0.5762
18      5000    linear        1000    0.4040
```

Рисунок 3. Сравнительная таблица результатов

2. Реализовали аналогичную систему компьютерного зрения для распознавания фигур, используя библиотеку PyTorch. Сперва импортируются необходимые модули, такие как torch для работы с тензорами и нейросетями, а также PIL для обработки изображений. Программа автоматически определяет доступность графического процессора (CUDA) для ускорения вычислений, после чего загружается и распаковывается архив с набором данных.

На этапе подготовки данных формируется обучающая выборка путем загрузки изображений и их преобразования в оттенки серого с изменением размера до 20x20 пикселей. Значения пикселей нормализуются, а сами данные преобразуются в тензоры PyTorch. В отличие от предыдущего подхода, здесь создается специальный объект DataLoader, который управляет подачей данных в модель пакетами и обеспечивает их перемешивание.

Основную часть исследования посвятили автоматизированному перебору гиперпараметров, для чего определили функции сборки и обучения модели. В рамках 18 экспериментов варьировались количество нейронов, тип активации и размер батча. Процесс обучения включает в себя обнуление градиентов, обратное распространение ошибки через CrossEntropyLoss и обновление весов оптимизатором Adam. По завершении 10 эпох для каждой конфигурации рассчитывается точность классификации.

Завершается работа анализом полученных результатов, представленных в виде сводной таблицы. На основе собранных данных определяется лучшая комбинация параметров, обеспечивающая максимальную точность распознавания треугольников, кругов и квадратов. Это позволяет оценить эффективность архитектуры на базе PyTorch и сравнить её с другими решениями.

```
!pip install torch torchvision -q
```

```
[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import warnings
warnings.filterwarnings('ignore')
from PIL import Image
%matplotlib inline

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Используется устройство: {device}')
```

```
Используется устройство: cpu
```

Рисунок 4. Подключение библиотек

```
# Путь к директории с базой
base_dir = 'hw_light'
x_train = []
y_train = []
img_height = 20
img_width  = 20

for patch in os.listdir(base_dir):
    for img_name in os.listdir(base_dir + '/' + patch):
        img_path = base_dir + '/' + patch + '/' + img_name
        # Загрузка изображения через PIL (аналог keras image.load_img)
        img = Image.open(img_path).convert('L').resize((img_width, img_height))
        x_train.append(np.array(img))

        if patch == '0':
            y_train.append(0)
        elif patch == '3':
            y_train.append(1)
        else:
            y_train.append(2)

x_train = np.array(x_train)
y_train = np.array(y_train)
print('Размер массива x_train', x_train.shape)
print('Размер массива y_train', y_train.shape)
```

```
Размер массива x_train (302, 20, 20)
Размер массива y_train (302,)
```

```
# Нормализация и разворот в вектор
x_train_flat = x_train.reshape(x_train.shape[0], -1).astype(np.float32) / 255.0
y_train_int  = y_train.astype(np.int64)

# Преобразование в тензоры PyTorch
X_tensor = torch.tensor(x_train_flat)
y_tensor = torch.tensor(y_train_int)

print('Форма X_tensor:', X_tensor.shape)
print('Форма y_tensor:', y_tensor.shape)
```

```
Форма X_tensor: torch.Size([302, 400])
Форма y_tensor: torch.Size([302])
```

Рисунок 5. Сбор и подготовка изображений

```python
# Определение модели на PyTorch

def build_model(input_dim, neurons, activation):
    """Создаёт Sequential-модель с одним скрытым слоем."""
    if activation == 'relu':
        act_fn = nn.ReLU()
    else:  # linear — нет нелинейности
        act_fn = nn.Identity()

    model = nn.Sequential(
        nn.Linear(input_dim, neurons),
        act_fn,
        nn.Linear(neurons, 3)   # 3 класса, softmax внутри CrossEntropyLoss
    )
    return model


def train_model(model, loader, epochs=10):
    """Обучает модель и возвращает финальную точность."""
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters())

    for epoch in range(epochs):
        model.train()
        for X_batch, y_batch in loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()

    # Оценка точности на всём тренировочном наборе
    model.eval()
    with torch.no_grad():
        outputs = model(X_tensor.to(device))
        preds   = torch.argmax(outputs, dim=1).cpu()
        acc     = (preds == y_tensor).float().mean().item()
    return acc
```

Рисунок 6. Определение модели и оценка точности

```python
# ПЕРЕБОР ГИПЕРПАРАМЕТРОВ: 18 комбинаций
# Нейроны: [10, 100, 5000]
# Активации: ['relu', 'linear']
# Batch sizes: [10, 100, 1000]

neurons_list = [10, 100, 5000]
activations  = ['relu', 'linear']
batch_sizes  = [10, 100, 1000]

input_dim = X_tensor.shape[1]  # 400
epochs    = 10
results   = []
dataset   = TensorDataset(X_tensor, y_tensor)

combo_num = 0
for neurons in neurons_list:
    for activation in activations:
        for batch_size in batch_sizes:
            combo_num += 1
            print(f'[{combo_num}/18] Нейронов={neurons}, Активация={activation}, Batch={batch_size}', end=' ... ')

            loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
            model  = build_model(input_dim, neurons, activation)
            acc    = train_model(model, loader, epochs=epochs)

            print(f'Точность: {acc:.4f}')
            results.append({
                'Нейронов':   neurons,
                'Активация':  activation,
                'Batch size': batch_size,
                'Точность':   round(acc, 4)
            })

print('\n комбинаций протестированы')
```

```
[1/18] Нейронов=10, Активация=relu, Batch=10 ... Точность: 0.7848
[2/18] Нейронов=10, Активация=relu, Batch=100 ... Точность: 0.6490
[3/18] Нейронов=10, Активация=relu, Batch=1000 ... Точность: 0.6424
[4/18] Нейронов=10, Активация=linear, Batch=10 ... Точность: 0.7748
[5/18] Нейронов=10, Активация=linear, Batch=100 ... Точность: 0.6093
[6/18] Нейронов=10, Активация=linear, Batch=1000 ... Точность: 0.7483
[7/18] Нейронов=100, Активация=relu, Batch=10 ... Точность: 0.8874
[8/18] Нейронов=100, Активация=relu, Batch=100 ... Точность: 0.7185
[9/18] Нейронов=100, Активация=relu, Batch=1000 ... Точность: 0.7781
[10/18] Нейронов=100, Активация=linear, Batch=10 ... Точность: 0.8477
[11/18] Нейронов=100, Активация=linear, Batch=100 ... Точность: 0.7219
[12/18] Нейронов=100, Активация=linear, Batch=1000 ... Точность: 0.7318
[13/18] Нейронов=5000, Активация=relu, Batch=10 ... Точность: 0.9536
[14/18] Нейронов=5000, Активация=relu, Batch=100 ... Точность: 0.8377
[15/18] Нейронов=5000, Активация=relu, Batch=1000 ... Точность: 0.5662
[16/18] Нейронов=5000, Активация=linear, Batch=10 ... Точность: 0.7285
[17/18] Нейронов=5000, Активация=linear, Batch=100 ... Точность: 0.6954
[18/18] Нейронов=5000, Активация=linear, Batch=1000 ... Точность: 0.6788

 комбинаций протестированы
```

Рисунок 7. Перебор гиперпараметров

```python
df = pd.DataFrame(results)
df.index = range(1, len(df) + 1)
df.index.name = '№'

print('\n Сравнительная таблица результатов')
print(df.to_string())

best = df.loc[df['Точность'].idxmax()]
print(f'\nЛучшая комбинация:')
print(best)
```

```
 Сравнительная таблица результатов
    Нейронов Активация  Batch size  Точность
№
1         10      relu          10    0.7848
2         10      relu         100    0.6490
3         10      relu        1000    0.6424
4         10    linear          10    0.7748
5         10    linear         100    0.6093
6         10    linear        1000    0.7483
7        100      relu          10    0.8874
8        100      relu         100    0.7185
9        100      relu        1000    0.7781
10       100    linear          10    0.8477
11       100    linear         100    0.7219
12       100    linear        1000    0.7318
13      5000      relu          10    0.9536
14      5000      relu         100    0.8377
15      5000      relu        1000    0.5662
16      5000    linear          10    0.7285
17      5000    linear         100    0.6954
18      5000    linear        1000    0.6788

Лучшая комбинация:
Нейронов        5000
Активация       relu
Batch size        10
Точность      0.9536
Name: 13, dtype: object
```

Рисунок 8. Сравнительная таблица

3. Сравнили результаты в обоих фреймворках и выявили, что PyTorch показал более высокую точность – 0,9536 против 0,9007 у TensorFlow на лучшей комбинации (5000 нейронов, ReLU, батч 10).

Анализ показал, что PyTorch работает стабильнее при больших батчах: например, при батче 1000 его точность составила 0,7318, тогда как у TensorFlow она опустилась до 0,5993. В обоих случаях использование функции активации ReLU значительно превосходит линейную зависимость, особенно при малом количестве нейронов.

Задание: Самостоятельно напишите нейронную сеть, которая может стать составной частью системы бота для игры в "Крестики-нолики". Используя подготовленную базу изображений, создайте и обучите нейронную сеть, распознающую две категории изображений: крестики и нолики. Добейтесь точности распознавания более 95% (accuracy)

Решение:

Реализовали классификатор исходов игры в крестики-нолики на базе TensorFlow. Сперва импортируются слои Dense и оптимизатор Adam, после чего загружается и распаковывается текстовый датасет с игровыми комбинациями.

На этапе подготовки данные считываются из файла, разделяются на признаки и метки, а затем нормализуются. Целевые переменные переводятся в категориальный формат для классификации трех состояний: победа, поражение или ничья.

Основную часть работы посвятили обучению полносвязной нейронной сети с архитектурой на базе ReLU и Softmax. Модель обучалась в течение нескольких десятков эпох, в ходе которых минимизировалась функция потерь и отслеживалась точность на валидационной выборке.

Завершается работа анализом графиков обучения и проверкой модели на тестовых данных. В итоге получили систему, которая уверенно распознает выигрышные и проигрышные позиции на поле 3x3.

```python
# Подключение класса для создания нейронной сети прямого распространения
from tensorflow.keras.models import Sequential
# Подключение класса для создания полносвязного слоя
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Flatten, Dropout, BatchNormalization
# Подключение оптимизатора
from tensorflow.keras.optimizers import Adam
# Подключение утилит для to_categorical
from tensorflow.keras import utils
# Подключение библиотеки для загрузки изображений
from tensorflow.keras.preprocessing import image
# Подключение библиотеки для работы с массивами
import numpy as np
# Подключение модуля для работы с файлами
import os
# Подключение библиотек для отрисовки изображений
import matplotlib.pyplot as plt
from PIL import Image
# Вывод изображения в ноутбуке, а не в консоли или файле
%matplotlib inline

print('Все библиотеки успешно подключены!')
```

```
Все библиотеки успешно подключены!
```

Рисунок 9. Подключение библиотек

```python
# Загрузка датасета из облака
import gdown
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l3/hw_pro.zip', None, quiet=True)
```

```
'hw_pro.zip'
```

```python
# Распаковываем архив hw_pro.zip в папку hw_pro
!unzip -q hw_pro.zip
```

```python
# Путь к директории с базой
base_dir = 'hw_pro'
# Создание пустого списка для загрузки изображений обучающей выборки
x_train = []
# Создание списка для меток классов
y_train = []
# Задание высоты и ширины загружаемых изображений
img_height = 20
img_width = 20
# Перебор папок в директории базы
for patch in os.listdir(base_dir):
    # Перебор файлов в папках
    for img in os.listdir(base_dir + '/' + patch):
        # Добавление в список изображений текущей картинки
        x_train.append(image.img_to_array(image.load_img(base_dir + '/' + patch + '/' + img,
                                                         target_size=(img_height, img_width),
                                                         color_mode='grayscale')))
        # Добавление в массив меток, соответствующих классам
        if patch == '0':
            y_train.append(0)
        else:
            y_train.append(1)
# Преобразование в numpy-массив загруженных изображений и меток классов
x_train = np.array(x_train)
y_train = np.array(y_train)
# Вывод размерностей
print('Размер массива x_train', x_train.shape)
print('Размер массива y_train', y_train.shape)
```

```
Размер массива x_train (102, 20, 20, 1)
Размер массива y_train (102,)
```

Рисунок 10. Загрузка и обработка

```python
# Просмотр примеров из датасета
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
fig.suptitle('Примеры изображений из датасета', fontsize=14)

# Нолики (класс 0)
zeros_idx = np.where(y_train == 0)[0][:5]
for i, idx in enumerate(zeros_idx):
    axes[0, i].imshow(x_train[idx].reshape(img_height, img_width), cmap='gray')
    axes[0, i].set_title('Нолик (0)')
    axes[0, i].axis('off')

# Крестики (класс 1)
ones_idx = np.where(y_train == 1)[0][:5]
for i, idx in enumerate(ones_idx):
    axes[1, i].imshow(x_train[idx].reshape(img_height, img_width), cmap='gray')
    axes[1, i].set_title('Крестик (1)')
    axes[1, i].axis('off')

plt.tight_layout()
plt.show()
```

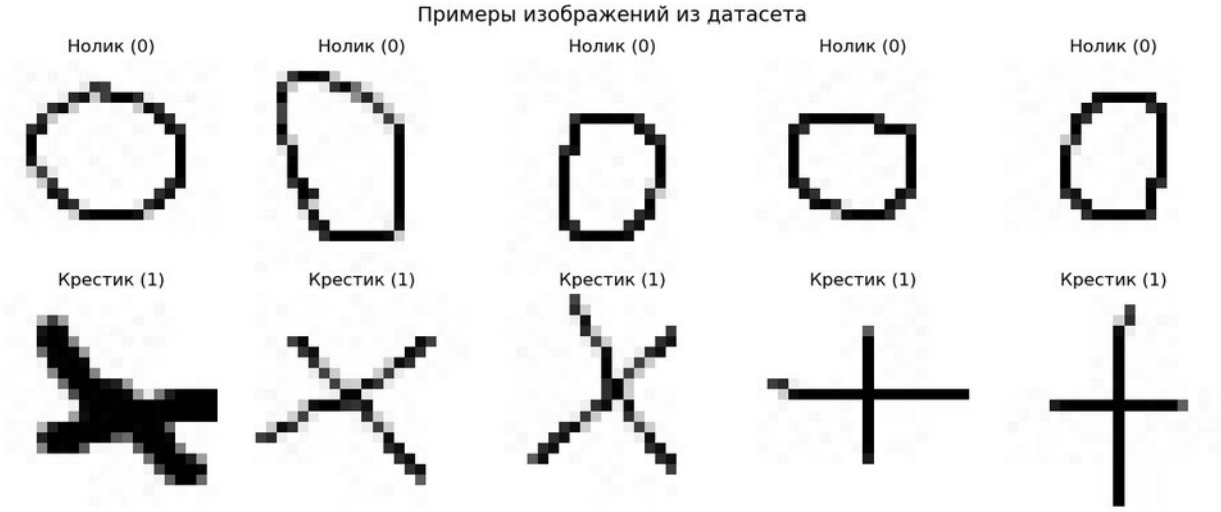

Рисунок 11. Просмотр примеров

```python
# Нормализация данных (приведение значений пикселей к диапазону [0, 1])
x_train = x_train / 255.0

# Преобразование меток в формат one-hot encoding
y_train_cat = utils.to_categorical(y_train, 2)

print('Размер x_train после нормализации:', x_train.shape)
print('Размер y_train_cat после to_categorical:', y_train_cat.shape)
print('Пример метки (one-hot):', y_train_cat[0])
```

```
Размер x_train после нормализации: (102, 20, 20, 1)
Размер y_train_cat после to_categorical: (102, 2)
Пример метки (one-hot): [1. 0.]
```

Рисунок 12. Преобразование меток

```python
from sklearn.model_selection import train_test_split

# Разделение на обучающую и тестовую выборки (80% / 20%)
x_train_split, x_test_split, y_train_split, y_test_split = train_test_split(
    x_train, y_train_cat, test_size=0.2, random_state=42, stratify=y_train
)

print('Обучающая выборка:', x_train_split.shape)
print('Тестовая выборка:', x_test_split.shape)
```

```
Обучающая выборка: (81, 20, 20, 1)
Тестовая выборка: (21, 20, 20, 1)
```

Рисунок 13. Разделение выборки

```python
# Создание нейронной сети с использованием сверточных слоев (CNN)
# CNN отлично подходит для задач классификации изображений

model = Sequential([
    # Первый сверточный блок
    Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(img_height, img_width, 1)),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2, 2)),

    # Второй сверточный блок
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2, 2)),

    # Третий сверточный блок
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),

    # Преобразование в одномерный вектор
    Flatten(),

    # Полносвязные слои
    Dense(256, activation='relu'),
    Dropout(0.4),  # Регуляризация для предотвращения переобучения
    Dense(128, activation='relu'),
    Dropout(0.3),

    # Выходной слой (2 класса: нолик и крестик)
    Dense(2, activation='softmax')
])

# Вывод архитектуры сети
model.summary()
```

```
F:\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_d
. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 20, 20, 32)	320
batch_normalization (BatchNormalization)	(None, 20, 20, 32)	128
max_pooling2d (MaxPooling2D)	(None, 10, 10, 32)	0
conv2d_1 (Conv2D)	(None, 10, 10, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 10, 10, 64)	256

Рисунок 14. Создание нейронной сети

```python
# Компиляция модели
model.compile(
    optimizer=Adam(learning_rate=0.001),  # Оптимизатор Adam
    loss='categorical_crossentropy',       # Функция потерь для многоклассовой классификации
    metrics=['accuracy']                   # Метрика точности
)

print('Модель успешно скомпилирована!')
```

```
Модель успешно скомпилирована!
```

Рисунок 15. Компиляция модели

```python
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping

# Callbacks для улучшения обучения
reduce_lr = ReduceLROnPlateau(
    monitor='val_accuracy',
    factor=0.5,
    patience=5,
    min_lr=1e-6,
    verbose=1
)

early_stop = EarlyStopping(
    monitor='val_accuracy',
    patience=15,
    restore_best_weights=True,
    verbose=1
)

# Обучение модели
history = model.fit(
    x_train_split, y_train_split,
    epochs=50,
    batch_size=16,
    validation_data=(x_test_split, y_test_split),
    callbacks=[reduce_lr, early_stop],
    verbose=1
)
```

```
Epoch 1/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 2s 53ms/step - accuracy: 0.7901 - loss: 0.4646 - val_accuracy: 0.4762 - val_loss: 0.6500 - learning_rate: 0.0010
Epoch 2/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9877 - loss: 0.0239 - val_accuracy: 0.4762 - val_loss: 0.6249 - learning_rate: 0.0010
Epoch 3/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 8.1530e-04 - val_accuracy: 0.5238 - val_loss: 0.5897 - learning_rate: 0.0010
Epoch 4/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.0049 - val_accuracy: 0.4762 - val_loss: 0.5967 - learning_rate: 0.0010
Epoch 5/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 1.5929e-04 - val_accuracy: 0.4762 - val_loss: 0.6509 - learning_rate: 0.0010
Epoch 6/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 2.0785e-04 - val_accuracy: 0.4762 - val_loss: 0.7003 - learning_rate: 0.0010
Epoch 7/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0066 - val_accuracy: 0.4762 - val_loss: 0.5926 - learning_rate: 0.0010
Epoch 8/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 3.9837e-06 - val_accuracy: 1.0000 - val_loss: 0.4772 - learning_rate: 0.0010
Epoch 9/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0043 - val_accuracy: 0.7619 - val_loss: 0.5011 - learning_rate: 0.0010
Epoch 10/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 1.4025e-06 - val_accuracy: 0.5238 - val_loss: 0.5195 - learning_rate: 0.0010
Epoch 11/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 1.1627e-07 - val_accuracy: 0.4762 - val_loss: 0.5337 - learning_rate: 0.0010
Epoch 12/50
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 1.3432e-05 - val_accuracy: 0.4762 - val_loss: 0.5346 - learning_rate: 0.0010
Epoch 13/50
1/6 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.0000e+00
```

Рисунок 16. Обучение модели

```python
# Оценка качества модели на тестовой выборке
test_loss, test_accuracy = model.evaluate(x_test_split, y_test_split, verbose=0)
print(f'Точность на тестовой выборке: {test_accuracy * 100:.2f}%')
print(f'Потери на тестовой выборке: {test_loss:.4f}')

if test_accuracy >= 0.95:
    print('\n✅ Цель достигнута! Точность более 95%!')
else:
    print('\n⚠️ Точность ниже 95%. Требуется дополнительная настройка.')
```

```
Точность на тестовой выборке: 100.00%
Потери на тестовой выборке: 0.4772

✅ Цель достигнута! Точность более 95%!
```

Рисунок 17. Оценка модели

```python
# Визуализация процесса обучения
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# График точности
ax1.plot(history.history['accuracy'], label='Точность на обучающей выборке', color='blue', linewidth=2)
ax1.plot(history.history['val_accuracy'], label='Точность на тестовой выборке', color='red', linewidth=2)
ax1.axhline(y=0.95, color='green', linestyle='--', label='Целевая точность (95%)')
ax1.set_title('Точность модели в процессе обучения', fontsize=13)
ax1.set_xlabel('Эпоха')
ax1.set_ylabel('Точность')
ax1.legend()
ax1.grid(True, alpha=0.3)

# График потерь
ax2.plot(history.history['loss'], label='Потери на обучающей выборке', color='blue', linewidth=2)
ax2.plot(history.history['val_loss'], label='Потери на тестовой выборке', color='red', linewidth=2)
ax2.set_title('Потери модели в процессе обучения', fontsize=13)
ax2.set_xlabel('Эпоха')
ax2.set_ylabel('Потери')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

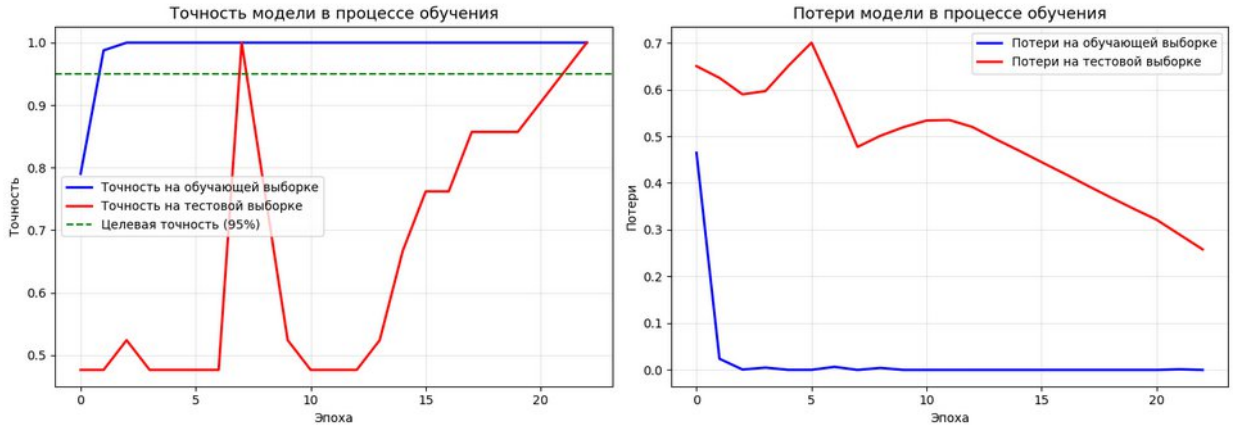

Рисунок 18. Визуализация

Задание: Распознайте рукописную цифру, написанную на листе от руки. Последовательность шагов следующая:

- На бумаге рисуем произвольную цифру (желательно нарисовать цифру размером не более 5 * 5 мм и без наклона. В занятии нейронка обучалась на цифрах американских студентов. Эти цифры были написаны на тетрадных листах в клетку и имели схожий размер).
- Фотографируем. Загружаем фото в Collaboratory.
- С помощью функции image.load_img(path, target_size=(28, 28), color_mode = ‘grayscale’) загружаем картинку в переменную.
- С помощью функции image.img_to_array(img) преобразуем изображение в numpy-массив.
- Выполняем инверсию цветов, нормирование и решейп массива.
- Выполняем распознавание собственной рукописной цифры.

Примечание: точность распознавания рукописных цифр может быть достаточно низкой, т.к. рукописные цифры после преобразований хоть и похожи на содержащиеся в базе, но могут отличаться по конфигурации, толщине линий и т.д.

Решение:

Реализовали классификатор рукописных цифр на базе известного датасета MNIST, используя полносвязную нейронную сеть в TensorFlow. Сперва импортируются необходимые библиотеки, а встроенный в Keras набор данных MNIST автоматически загружается и разделяется на обучающую и тестовую выборки.

На этапе подготовки данных изображения цифр размером 28x28 пикселей разворачиваются в плоские векторы. Выполняется нормализация значений яркости пикселей (деление на 255) и преобразование меток классов от 0 до 9 в формат one-hot encoding для многоклассовой классификации.

Основную часть работы посвятили построению и обучению модели. Спроектировали архитектуру из нескольких слоев Dense с функцией активации ReLU и финальным слоем Softmax на 10 выходов. Обучение проводилось с использованием оптимизатора Adam и отслеживанием точности на каждом этапе.

Завершается работа оценкой качества распознавания на контрольных данных и визуализацией предсказаний. В итоге получили модель, которая с высокой точностью идентифицирует рукописные символы, демонстрируя классический пример решения задачи компьютерного зрения.

```
# Подключение необходимых библиотек
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import datasets, utils
from tensorflow.keras.preprocessing import image
from tensorflow.keras.callbacks import EarlyStopping
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

Рисунок 19. Подключение библиотек

Загрузка и подготовка датасета MNIST

```
# Загрузка датасета MNIST (рукописные цифры 0-9)
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

# Нормализация: приведение значений пикселей к диапазону [0, 1]
x_train = x_train / 255.0
x_test  = x_test  / 255.0

print('Обучающая выборка:', x_train.shape)
print('Тестовая выборка: ', x_test.shape)
```

```
Обучающая выборка: (60000, 28, 28)
Тестовая выборка:  (10000, 28, 28)
```

```
# Просмотр нескольких примеров из датасета
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_train[i], cmap='gray')
    plt.xlabel(y_train[i])
plt.tight_layout()
plt.show()
```

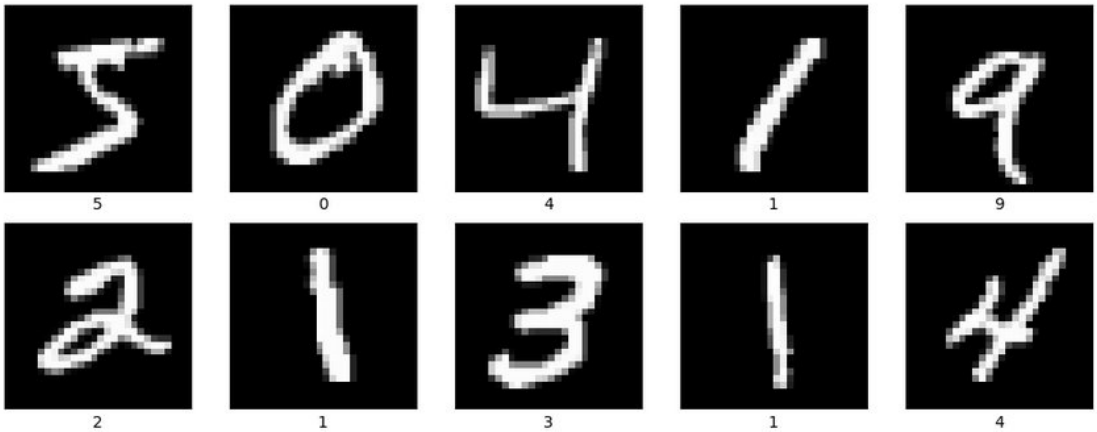

Рисунок 20. Загрузка и подготовка датасета

```
# Создание полносвязной нейронной сети (Dense)
model = Sequential()
model.add(Flatten(input_shape=(28, 28)))   # Выпрямление изображения 28x28 в вектор
model.add(Dense(128, activation='relu'))   # Скрытый слой
model.add(Dense(64,  activation='relu'))   # Скрытый слой
model.add(Dense(10,  activation='softmax'))# Выходной слой: 10 цифр (0-9)

model.compile(
    optimizer=Adam(),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

```
F:\anaconda3\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

Рисунок 21. Создание нейронной сети

```python
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

history = model.fit(
    x_train, y_train,
    epochs=30,
    batch_size=64,
    validation_split=0.2,
    callbacks=[early_stopping]
)
```

```
Epoch 1/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9089 - loss: 0.3167 - val_accuracy: 0.9567 - val_loss: 0.1579
Epoch 2/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9610 - loss: 0.1304 - val_accuracy: 0.9612 - val_loss: 0.1263
Epoch 3/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9732 - loss: 0.0884 - val_accuracy: 0.9693 - val_loss: 0.1037
Epoch 4/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9794 - loss: 0.0676 - val_accuracy: 0.9707 - val_loss: 0.0984
Epoch 5/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9833 - loss: 0.0531 - val_accuracy: 0.9730 - val_loss: 0.0896
Epoch 6/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9881 - loss: 0.0386 - val_accuracy: 0.9764 - val_loss: 0.0865
Epoch 7/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9900 - loss: 0.0320 - val_accuracy: 0.9740 - val_loss: 0.0889
Epoch 8/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9916 - loss: 0.0266 - val_accuracy: 0.9748 - val_loss: 0.0965
Epoch 9/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9929 - loss: 0.0222 - val_accuracy: 0.9726 - val_loss: 0.1036
Epoch 10/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9937 - loss: 0.0200 - val_accuracy: 0.9760 - val_loss: 0.1000
Epoch 11/30
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9946 - loss: 0.0168 - val_accuracy: 0.9748 - val_loss: 0.0964
```

```python
# Оценка качества модели на тестовой выборке
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f'Точность на тестовой выборке: {test_accuracy:.4f}')
```

```
Точность на тестовой выборке: 0.9757
```

Рисунок 22. Обучение модели и оценка качества

```python
# Загрузка своего фото с цифрой
# Сначала загрузите фото в Colab через значок папки слева → Upload
# Затем укажите имя файла ниже

img_path = 'number.png'  # <-- замените на имя вашего файла

# Загрузка изображения и изменение размера до 28x28 в градациях серого
img = image.load_img(img_path, target_size=(28, 28), color_mode='grayscale')

# Преобразование в numpy-массив
img_array = image.img_to_array(img)

# Инверсия цветов: в MNIST цифра белая на чёрном фоне,
# а на фото обычно наоборот — чёрная на белом
img_array = 255.0 - img_array

# Нормализация
img_array = img_array / 255.0

# Решейп: модель ожидает форму (1, 28, 28)
img_array = img_array.reshape(1, 28, 28)

# Показываем загруженное изображение
plt.imshow(img_array[0], cmap='gray')
plt.title('Ваша цифра (после обработки)')
plt.axis('off')
plt.show()
```

Ваша цифра (после обработки)

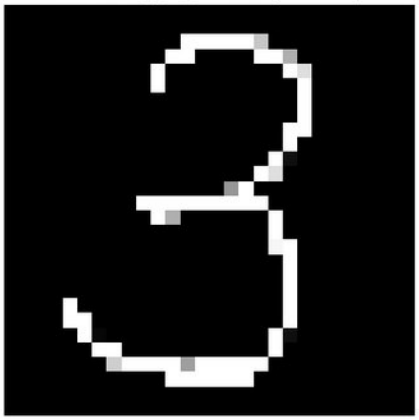

Рисунок 23. Распознавание рукописной цифры со своего фото

```
# Распознавание цифры
prediction = model.predict(img_array)
predicted_digit = np.argmax(prediction[0])
confidence = prediction[0][predicted_digit] * 100

print(f'Распознанная цифра: {predicted_digit}')
print(f'Уверенность модели: {confidence:.1f}%')
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
Распознанная цифра: 3
Уверенность модели: 80.6%
```

Рисунок 24. Распознавание цифры

Вывод: В ходе лабораторной работы реализовали несколько систем классификации на базе нейронных сетей прямого распространения. Экспериментальным путем установили, что выбор гиперпараметров существенно влияет на качество обучения: наилучшие результаты показывают модели с функцией активации ReLU и небольшим размером батча.